

CSDS 233 Programming Assignment 4 Priority Queues and Simulations
--

Introduction:

In role-playing games, player-characters often interact with computer-generated characters. To make this interaction look realistic, a number of techniques are used. For example, suppose that your group of ten characters arrive at a wizard's cave and have brought the wizard a magic ruby. As a reward, the wizard has offered to teach each member of your group a new spell. The computer program *could* take the player-characters in the order in which they walked into the wizard's cave (a standard First-In, First-Out queue), but it would be more realistic if the wizard mumbles something about wasting less time if she takes the characters in reverse order of how long it will take to teach each character their spell, and then interacts with each character in that sequence.

In this assignment we will build a simulation of what happens when characters wait in a queue for access to a wizard's cave for help in learning new spells. This scenario is a simple example of scheduling access to a resource through a priority queue and is extremely common in organizing the activities in a computer system or in a network. To just give one example, whenever you run a program, your running program must wait in a job queue for access to a CPU, must wait in a disk queue when reading from the disk, and finally must wait in the printer queue before you can obtain a hard copy of your work.

Wizard's Cave Simulation:

We will simulate what happens when characters arrive at a wizard's cave to ask for help. The simulation will have the following features:

- Time is measured in seconds elapsed from the time the wizard becomes available.
- A sequence of characters $S_0, S_1, \dots$ arrive with a request for the wizard to teach them a particular spell; each character S_i arrives at the cave at elapsed time T_i , and requires exactly R_i seconds of the wizard's time for help with their spell; when they arrive at the cave, they wait in a queue until they get access to the wizard; for simplicity we will assume that no two characters ever arrive at the cave at the exact same time.
- The wizard wants to give characters the best possible help, so she allows only one character in her cave at a time, and once a character S_i enters her cave, she gives the character here undivided attention for R_i seconds, teaching them the entire spell all in one session, so that no character has to go back and wait in the queue more than once.
- The wizard is very conscientious and very efficient; when a character finishes, if the queue is non-empty, the wizard instantly invites the first character in the queue into her cave for help; it takes 0 seconds for a character to leave and the next character to enter and begin getting help; if there are no characters in the queue, the wizard simply waits until a character arrives, and then instantly invites them into her cave.

An important design consideration is how to organize the queue (which is really a priority queue). The default for most waiting queues is **First-Come-First-Served** (FCFS). However, the wizard notices that sometimes a character requesting to be taught a very short spell (e.g., the short but quite useful "insect killer" spell) must wait behind a

character with a very long spell (the “ox-cart driving” spell, which takes a long time to learn – and even then the character still has to go pass the driving test before they can get their ox-cart license). One day, while waiting in the Express Checkout line at the farmer’s market in the middle of the village square, the wizard ponders how she might avoid this problem. She decides (while paying for a can of dragon food and a gallon of bat’s blood) to try using a policy of **Shortest-Request-First** (SRF); that is, whenever selecting a new character S_i from the queue, she should select the character whose request R_i is shortest. Ties may be broken arbitrarily. She decides she will try both and see which is better.

But how should she decide which is better? From an individual character's point of view, the important thing is to spend as little time waiting in the queue as possible; thus the wizard decides that she should use the strategy which results in the **smallest average wait time** for all characters who request his help.

Suppose we have character 0 arriving at time 0 with a request for 30 seconds of help, character 1 arriving at time 10 with a request time of 20 seconds, and character 2 arriving at 30 seconds with a request time of 10 seconds. We could chart the progress of these two policies as follows, where “++++” indicates time waiting in the queue, and “%%%%%%%%” represents time learning a spell:

FCFS:

Time:	0	10	20	30	40	50	60
Character 0:	%%%%%%%%%						
Character 1:		+++++					
Character 2:							

SRF:

Time:	0	10	20	30	40	50	60
Character 0:	%%%%%%%%%						
Character 1:		+++++					
Character 2:							

The average wait time is the sum of the seconds spent in the queue (all the +'s) divided by the number of characters, i.e., $(20+20)/3 = 13.333$ for FCFS and $30/3 = 10$ for SRF. Clearly, SRF is better in this case. The wizard decides to implement the SRF strategy, the player-characters move through the queue more quickly, and the wizard becomes even more beloved than she previously was.

Assignment:

Test files:

Some initial test files for the assignment are on Canvas (please **be sure to test your implementation with more than just these files**):

test.txt
error.txt
characters.txt

Your first task in this assignment will be to read an input file representing a sequence of characters. Each line in the file contains two pieces of information about a particular character: his or her *arrival time* into the queue and his or her *request time* - the amount of time it will take the wizard to teach that character his or her spell.

Second, you will calculate the average wait time under each of these two strategies.

Third, in order to help understand the process (and to debug), you will also provide a verbose trace so that you can see what events happen at which times during the simulation. A sample session for illustration purposes may be useful. If the scenario given above were simulated, we would get the following:

```
test.txt                                // This is an input file
0 30
10 20
30 10
```

Example run1:

```
Enter Queueing Style: fcfs
Enter input file: test.txt
```

Time	Event
----	-----
0	Character 0 enters queue
0	Character 0 enters cave
10	Character 1 enters queue
30	Character 2 enters queue
30	Character 0 leaves cave
30	Character 1 enters cave
50	Character 1 leaves cave
50	Character 2 enters cave
60	Character 2 leaves cave

Simulation over...

Average wait time: 13.3333

Example run2:

```
Enter Queueing Style: srf
Enter input file: test.txt
```

Time	Event
----	-----
0	Character 0 enters queue
0	Character 0 enters cave
10	Character 1 enters queue
30	Character 2 enters queue
30	Character 0 leaves cave
30	Character 2 enters cave
40	Character 2 leaves cave
40	Character 1 enters cave
60	Character 1 leaves cave

Simulation over...

Average wait time: 10

Details of the Implementation:

Please take careful note of the following features which must be part of your implementation:

1. As shown in the sample session, your program must ask the user for the queuing policy and the name of the input file. The input file will have a line for each character, containing two integers (the arrival time and the request time). You will not know how many lines there are in the file, but you can be assured that all the arrival times are distinct and in ascending order.
2. You will need, among others, the following data structures:
 - A **character class** which contains a priority, arrival time, request time, departure time, and character number (all integers); the departure time is the time at which the character leaves the wizard's cave, and is calculated when the character enters the wizard's cave; you will also need to write the **compareTo** function for the character class so that they can be compared based on priority for manipulation in the priority queue.
 - A variable for the next character to enter the queue (it's just read from the input file, but not in the queue yet);
 - A variable for the character currently in the wizard's cave;
 - A priority queue of waiting characters, organized as a **min-heap** (you can use Java's **priority queue collection**) which stores the smallest priority ranking at the top so that removing the top element gives you a minimal element (this is a min-heap); in the case of FCFS the priority ranking is simply the arrival time, and in the case of SRF the priority ranking is the request time; to avoid using two different implementations, you should simply use the variable **priority** variable in each character to order the queue; in the case of FCFS you assign the arrival time to this variable, and in the case of SRF you use the request time; again, you can/should use Java's priority queue collection to solve this problem
3. You will use a simple simulation technique called "time stepping." Your basic algorithm should be something very close to the following:

```
read the next character from the input file (if such exists);

for(time = 0; (next character exists OR heap not empty
              OR a character is in the wizard's cave); time++) {

    if( time == arrival time of next character if such exists ) {
        Insert this character into the heap;
        Print out this event;
        Read the next character from the input file (if such exists);
    }

    if( time == departure time of a character in the cave ) {
        Remove the character from the cave;
        Print out this event;
    }

    if( no character in cave AND heap is not empty ) {
```

```

        Remove a character from heap and put him/her in the cave;
        Print out this event;
    }
}

```

This pseudo-code leaves the matter of calculating the character number, the departure time, and average wait time intentionally unspecified. You should follow the sample session as closely as possible.

Error Checking:

You should also check for errors in input, as specified by the following example

Example run:

```

Enter Queueing Style: fcf // misspelled policy
Policy should be either fcfs or srf // remind user of usage
Enter Queueing Style: fcfs
Enter input file: nofile.txt // error in attempting to open file

Cannot open file nofile.txt

Contents of error.txt
0 30
10 20
30                                     // wrong number of integers in file

```

Example run:

```

Enter Queueing Style: srf
Enter input file: error.txt

Time      Event
----      -
0          Character 0 enters queue
0          Character 0 enters cave
10         Character 1 enters queue

Error in input file                // will find error when reading last
                                   // line

```

Analysis:

For the FCFS implementation we simply used our min-priority queue data structure and set the priority of each character to the character's arrival time. What advantages does this provide us? Would it be computationally more efficient to use a traditional FIFO queue for the FCFS situation? Explain why or why not.

Extra Credit Problem (5 points):

One problem with time stepping is that it is inefficient, since in many iterations of the loop, nothing is happening. A better way to calculate time is by need, i.e., during each iteration, figure out what the next event is, set the time variable to that time, and perform the next event(s). The simulation is over when there are no more events.

For example, if the next character has an arrival time of 30, and the character in the queue has a departure time of 25, then the next events are to remove the character from the queue and move a character from the heap to the queue if the heap is non-empty. Time in this model occurs in discrete steps corresponding to event times, and hence this is called "discrete event simulation." Rewrite your code using this model for an additional 5 points. Be sure to advertise this fact in the comments of your code so that we see it and give you credit for it!!

Individual Work:

Note it is expected that you do your own work on this. There should be no collaboration on this project with any of your classmates, any generative AI tool, or anything other than Java reference sites. If you have questions about the code, I encourage you to come talk with me or a TA.

Deliverables:

For this lab you need to turn in the following before 11:59 pm on the due date:

- Upload to Canvas all of the following:
 - Source Code files associated with this assignment, and only this assignment.
 - Lab Report which includes:
 - Analysis and Conclusions of what you learned from the lab.
 - Answers to the Analysis questions included above
 - Test Cases and Test Log
 - This should be a listing of the tests you tried on your code and a recording of the output.
 - Any references used

Grading:

This lab spell is worth 60 points distributed as follows:

Character Class Implementation	15
Correct Simulation FCFS	15
Correct Simulation SRF	15
Error Checking	10
Report including Test Log and Analysis	5